

Why RestroOS will not work as a restaurant POS

An honest evaluation of the current build against what a working restaurant till actually requires — and what to do about it.

Assessed build `30964dc` · Production environment · Reference site: Chai Chakhna Company, Kondapur (currently running PetPooja POS)

Verdict

RestroOS is a capable restaurant management platform with a POS-shaped screen attached. It is not a POS, and no amount of UI work will make it one.

Two of the blocking issues are architectural, not features: the system cannot operate without internet, and it cannot drive restaurant hardware. Either one alone disqualifies it from replacing a till in a working restaurant. Everything else in this document is secondary to those two.

Contents

- | | |
|-------------------------------------|--|
| 1 The two disqualifying constraints | 5 Reliability evidence from this build |
| 2 Missing service-floor operations | 6 What is genuinely strong |
| 3 Billing and compliance gaps | 7 Two viable strategies |
| 4 Cash, shifts and accountability | 8 Recommendation |

1 The two disqualifying constraints

1.1 The system stops working when the internet does **Blocking**

RestroOS is a browser application calling a serverless API backed by a remote database. There is no local persistence and no offline queue. When connectivity drops, staff cannot take an order, produce a bill, or record a payment.

Every established POS — PetPooja included — is local-first: it runs on the machine, writes to local storage, and reconciles with the server when the link returns. This is not a premium feature; it is the baseline assumption of the category.

WHY THIS IS DECISIVE

A restaurant cannot stop billing customers because the Wi-Fi dropped. There is no acceptable degraded mode: the queue does not pause, and the kitchen does not stop. A till that is unavailable for twenty minutes during dinner service is not a till that gets adopted, however good the rest of it is.

1.2 No hardware integration Blocking

A POS is fundamentally a hardware integration product with a user interface attached. RestroOS currently drives none of it:

DEVICE	CURRENT STATE
Thermal KOT printer	None. Kitchen tickets go through the browser print dialog, which is unusable at service speed. Real printers speak ESC/POS over USB or LAN.
Bill printer	Same limitation.
Cash drawer	No trigger. Drawers open on a pulse from the printer port.
Barcode scanner	Not handled.
Customer display	Not supported.

Without a kitchen printer the kitchen must watch a screen, which is a different operating model that most kitchens will not accept. A browser cannot reach these devices directly; this requires a native application or a local print agent.

2 Missing service-floor operations

These were identified by direct comparison with the PetPooja terminal running at the reference site. Each is used many times per shift.

CAPABILITY	SEVERITY	WHY IT MATTERS
Hold / park order	High	Start an order, park it, serve someone else, resume. Constant at a counter. RestroOS has no concept of an unsubmitted order — you either place it or lose it.
Move KOT / items	High	A party changes table; a dish goes to the wrong one; two tables merge. Orders cannot be moved between tables at all.
Table states	High	PetPooja distinguishes Blank / Running / Printed / Paid / Running KOT. RestroOS has available-or-not. The missing <i>Printed vs Paid</i> distinction is precisely the window in which cash goes unaccounted.
Split bill	High	Not supported. Routine for groups.
Item 86 (out of stock)	Medium	One tap to disable an item when an ingredient runs out. Currently only a catalogue edit buried in menu management.
KOT as an entity	High	A KOT is printed, added to later (printing only the delta), reprinted, or cancelled with a reason. RestroOS has orders that can print a ticket — not the same object, and kitchens run on it.
Elapsed time on tables	Medium	No indication of how long a table has been running. This is the primary signal a floor manager reads.

3 Billing and compliance gaps

3.1 Tax model is too simple for Indian F&B High

GST is hardcoded at 2.5% CGST + 2.5% SGST applied uniformly to every order. Real establishments need per-item tax slabs, packaging charges, service charge, delivery charge and round-off. There is no item-level tax field at all, so this cannot be configured — it is a schema change.

Also observed: CGST and SGST are rounded independently, so the two components can sum to marginally more or less than the intended 5%.

3.2 No split payment High

Payment method is stored as a single value — Cash, Card or UPI. Part-cash-part-UPI, extremely common in India, cannot be represented. This is a data-model limitation, not a UI one.

3.3 No void or refund flow High

Orders can be cancelled, but there is no void-with-reason, no partial refund, and no audit of who authorised either. Any business handling cash needs both, and auditors expect them.

3.4 No bill-level discount Medium

Discounts exist only as coupon codes. A manager cannot apply a one-off reduction to a specific bill, which is a routine goodwill action.

4 Cash, shifts and accountability

4.1 No shift management High

There is no open-shift or close-shift concept, no opening float, no cash-drawer reconciliation and no Z-report. At the end of the night nobody can answer what the till *should* contain.

4.2 Wrong authentication model for a shared terminal High

Staff sign in with restaurant slug, email and password, holding a seven-day session. A POS is shared by several people across a shift and uses short PIN entry with instant switching. More seriously, **orders carry no record of which staff member took them**, so voids, discounts and cancellations cannot be attributed to a person.

COMBINED EFFECT

No shift reconciliation plus no per-staff attribution means cash discrepancies cannot be traced to a person or a period. For an owner, this is often the single most important thing a POS provides.

5 Reliability evidence from this build

The following were found during a single day of work on the production system. They are included not as a list of bugs — all are now fixed — but as evidence about where the risk sits.

5.1 Stock silently failed to deduct Was live in production

Order status is permitted to jump several steps forward, but inventory consumption was triggered by an exact match on "Preparing". An order moved directly from Pending to Ready therefore deducted nothing and could reach Delivered having never touched stock.

Verified on production: an order taken Pending → Preparing wrote 6 ledger rows and moved all 6 ingredients; an identical order taken Pending → Ready changed 0 of 6 and wrote 0 rows.

5.2 Stock valuation was arithmetically wrong Was live in production

Negative balances were multiplied directly by unit cost, so drifting ingredients *subtracted* from the branch total and propagated into COGS and margin reporting.

Reported stock value was ₹225,247.95. Correct floored value was ₹92,938.99 — an overstatement of roughly ₹132,000, feeding profitability reports.

5.3 The POS screen has no automated test coverage High

`PosOrderBuilder.jsx` (900+ lines) and `PosTableGrid.jsx` carry zero tests. During this session three visual regressions reached production on that screen — including a syntax error that every test passed through, caught only by a build, and a clipped status label caught only by a photograph from a real device.

THE PATTERN THAT MATTERS

The screen a restaurant bills through is the least-tested code in the repository, and the failures observed were not caught by the existing verification pipeline. Test-and-build passing is not a sufficient release signal for this surface today.

6 What is genuinely strong

A fair assessment has to state what should not be discarded.

RECIPE-LEVEL INVENTORY

Automatic per-order stock deduction driven by menu-item recipes, written to an append-only ledger with prior/post balances, unit conversion guarded by unit type, idempotent consumption and reversal-on-cancellation. This is **better than PetPooja's inventory handling** and is the most defensible asset in the product.

MULTI-TENANT ARCHITECTURE AND PERMISSIONS

Tenant isolation is enforced at the query layer rather than the UI, permissions are re-read from the database on every request rather than trusted from the token, and privilege-escalation paths (staff and branch management) are deliberately non-delegable.

CUSTOMER STOREFRONT

The ordering experience, coupon handling and order tracking are solid, and this is a channel PetPooja charges separately for.

7 Two viable strategies

Option A — Stop positioning it as a POS

Reposition as restaurant management and online ordering: inventory, recipes, menu, reporting and customer storefront. **Integrate with PetPooja rather than compete with it**, consuming its order feed to drive the inventory engine.

- **Why it works:** plays directly to the strongest asset, and the inventory depth is a real wedge against incumbents whose stock handling is weaker.
- **Effort:** weeks. No architectural change required.
- **Cost:** gives up the till, which is where the recurring revenue and the customer lock-in sit.

Option B — Build an actual POS

A local-first native application — Electron or React Native with an embedded database and background sync — plus printer and drawer drivers, KOT lifecycle, shifts, split payments and PIN authentication.

- **Why it works:** it is the only path to replacing an incumbent till.
- **Effort:** months, and it is a different application from the current web build. The existing backend and inventory engine are reusable; the front end largely is not.
- **Cost:** substantial, and it competes head-on with entrenched, hardware-bundled products.

8 Recommendation

Pursue Option A now and treat Option B as a deliberate, separately-funded decision — not as an incremental continuation of the current codebase.

The reasoning is straightforward. The two blocking constraints cannot be closed by iterating on the present build; they require a different runtime. Meanwhile the inventory engine already outperforms the incumbent and is shippable today. Continuing to refine the POS screen inside a browser application spends effort on the one part of the product that cannot reach parity, while the genuinely differentiated part remains under-marketed.

WHAT NOT TO DO

Do not keep polishing the current POS screen. Its layout is materially better than it was, but a well-designed till that stops working when connectivity drops, cannot print a kitchen ticket, and cannot reconcile a cash drawer will not be adopted by a working restaurant — and the reference site will keep running PetPooja alongside it.

If the current screen must ship anyway

Ordered by value per unit of effort:

1. **Per-staff attribution on orders** and a basic shift open/close with cash count. Small change, disproportionate value to an owner.
2. **Table states** — add a bill-printed timestamp and a paid state; surface elapsed time on the floor view.
3. **Hold and Move KOT**. The two most-used operations that are entirely absent.
4. **Split payment** — schema change from a single payment field to payment lines.
5. **Automated tests for the POS screen** before any further UI change to it.
6. **A local print agent** for ESC/POS, as the narrowest possible step toward real hardware.

Assessment of build `30964dc`. Findings in §5 were observed on the production environment and have since been fixed. Comparison in §2 is against PetPooja POS as deployed at the reference site. Severity labels: **Blocking** prevents adoption outright; **High** is encountered every shift; **Medium** has a workaround.